

Nhập môn công nghệ phần mềm

Lập kế hoạch và kiểm thử

GV: ThS. Ngô Tiến Đức

Nội dung chính

- Lập kế hoạch và ước lượng
- Một số kỹ thuật ước lượng
- Kiểm thử phần mềm
- Kiểm thử thực thi
- Kiểm thử phi thực thi

Lập kế hoạch và ước lượng

- Việc lập kế hoạch được thực hiện liên tục trong suốt quá trình phát triển và bảo trì phần mềm
- Lên kế hoạch sơ bộ đầu dự án
- Kế hoạch chi tiết được lập sau khi đã có tài liệu đặc tả





- Văn bản cấp phép cho dự án và cấp quyền cho PM
- Xác định mục đích, mục tiêu sản phẩm và tiêu chí thành công
- Đưa ra ngân sách sơ bộ và các milestone
- Liệt kê các bên liên quan (stakeholder)
- Tổng quát, ngắn gọn, ít thay đổi



- Tài liệu chi tiết: Cách thực hiện, giám sát và kiểm soát dự án
- Lựa chọn quy trình phát triển
- Xác định chi tiết các đầu việc
- Cơ cấu nhân sự và phân công công việc
- Các công cụ và phương pháp kỹ thuật sẽ sử dụng
- Cập nhật thường xuyên

Lập kế hoạch và ước lượng

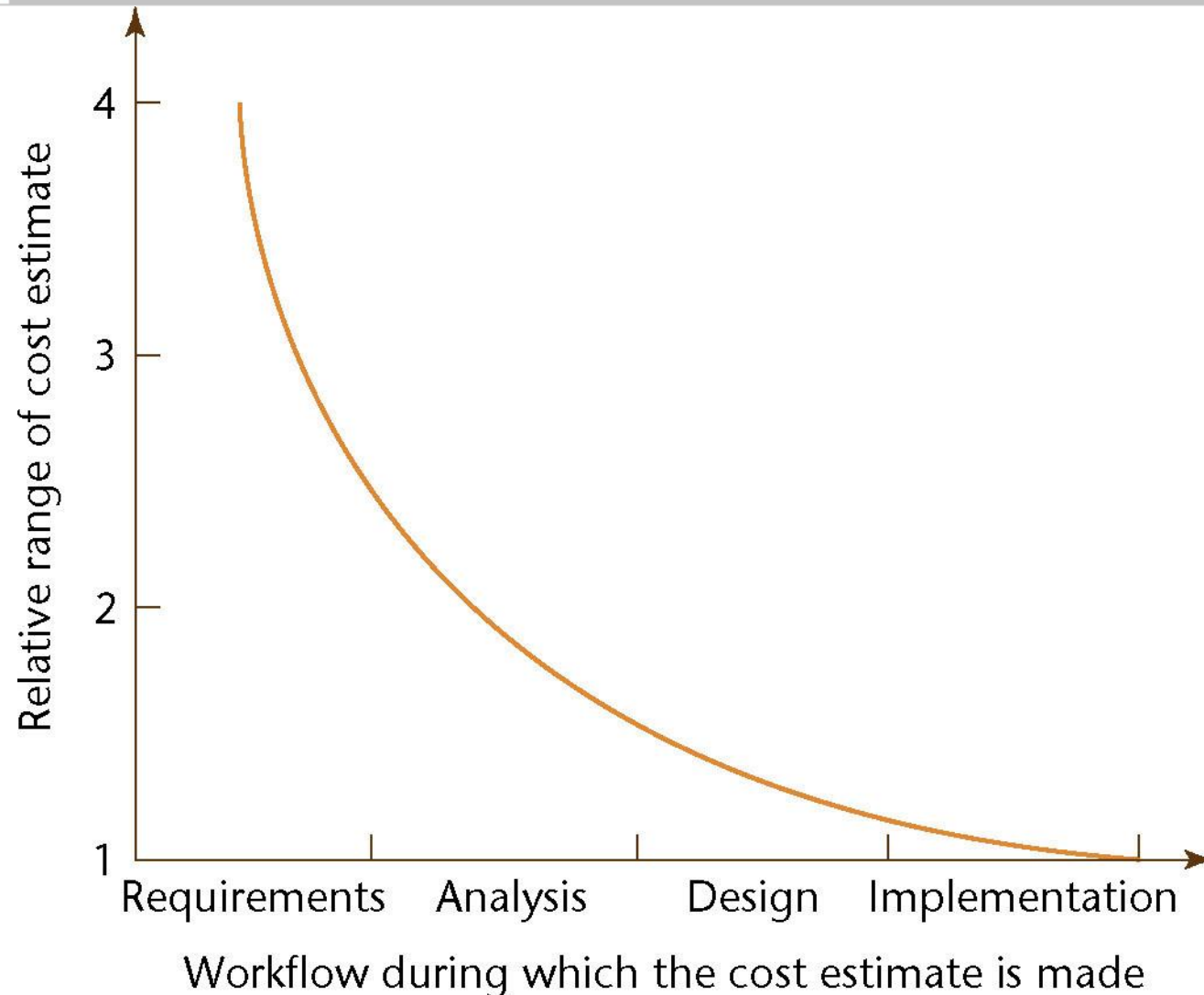
SPMP – Kế hoạch quản lý dự án phần mềm

- Việc quản lý được thực hiện trên nhiều khía cạnh



- Yếu tố then chốt cho quá trình lên kế hoạch: **Ước lượng**

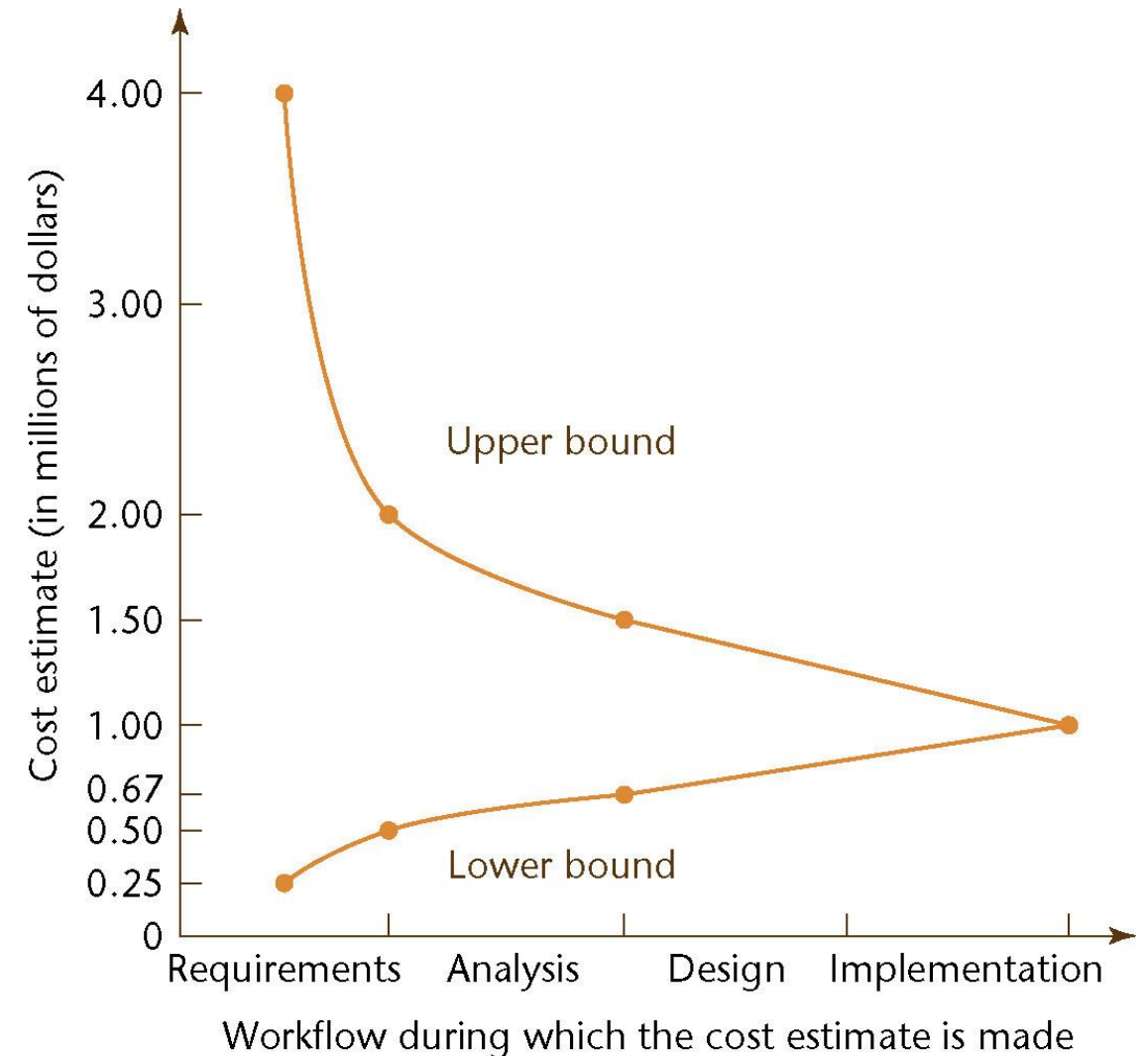
Lập kế hoạch và ước lượng



- Độ chính xác của ước lượng tăng dần theo tiến trình phát triển phần mềm
- Khách hàng thường yêu cầu dự toán kinh phí và thời hạn bàn giao sớm

Lập kế hoạch và ước lượng

- Ví dụ: Chi phí thực tế ~ 1 triệu đô
- Dữ liệu từ 1976
- Cone of uncertainty: "Old but gold"





- Nhà phát triển cần lưu ý:
 - Internal cost và external cost
 - Cân bằng khía cạnh kinh tế và tâm lý: Tránh giá quá cao hoặc quá thấp
 - Đảm bảo thời hạn bàn giao
 - Có rất nhiều yếu tố có thể ảnh hưởng đến quá trình ước lượng

Lập kế hoạch và ước lượng

- Ảnh hưởng của yếu tố con người đến quá trình ước lượng:
 - Khác biệt về hiệu năng làm việc [Sackman, 1968]:
 - Product size (6 : 1), product execution time (8 : 1), development time (9 : 1), coding time (18 : 1), **debugging time (28 : 1)**
 - Nhân sự rời dự án
- Yếu tố tiềm ẩn: Kích thước sản phẩm

Một số kỹ thuật ước lượng

Ước lượng thông qua số dòng code: LOC, KLOC, KDSI

- Không phải những gì viết ra đều có giá trị bàn giao cho khách hàng
- Phụ thuộc ngôn ngữ lập trình
- Không đủ tổng quát
- Số dòng code chính xác chỉ có thể xác định khi sản phẩm đã hoàn thành
-> Quá rủi ro

Một số kỹ thuật ước lượng

Ước lượng bằng FFP

- Dựa trên các thành phần: Files (F_i), Flows (F_l), Processes (P_r)
- Kích thước: $S = F_i + F_l + P_r$
- Chi phí: $C = b \times S$
 - Hằng số **b** (hoặc **d**): Thước đo năng suất
- Có thể tính toán sau khi hoàn thành việc thiết kế kiến trúc
- Chưa bao gồm CSDL

Một số kỹ thuật ước lượng

Ước lượng bằng Function Points

- Dựa trên số lượng inputs (Inp), outputs (Out), inquiries (Inq), master files (Maf), interfaces (Inf)
- Bước 1: Xác định unadjusted function points (UFP) cho từng thành phần

Component	Level of Complexity		
	Simple	Average	Complex
Input item	3	4	6
Output item	4	5	7
Inquiry	3	4	6
Master file	7	10	15
Interface	5	7	10

Một số kỹ thuật ước lượng

Ước lượng bằng Function Points

Bước 2: Tính toán technical complexity factor (TCF)

- 14 yếu tố ảnh hưởng kỹ thuật
- Mỗi yếu tố được đánh giá mức độ ảnh hưởng từ 0 đến 5
-> tính tổng -> degree of influence (*DI*)

$$TCF = 0.65 + 0.01 \times DI$$

1. Data communication
2. Distributed data processing
3. Performance criteria
4. Heavily utilized hardware
5. High transaction rates
6. Online data entry
7. End-user efficiency
8. Online updating
9. Complex computations
10. Reusability
11. Ease of installation
12. Ease of operation
13. Portability
14. Maintainability

Một số kỹ thuật ước lượng

Ước lượng bằng Function Points

- Bước 3: $FP = UFP \times TCF$
- Công thức đơn giản hóa (oversimplification):

$$FP = 4 \times Inp + 5 \times Out + 4 \times Inq + 10 \times Maf + 7 \times Inf$$



- Xem xét Intermediate COCOMO (Constructive Cost Model)
- Bước 1: Ước lượng KDSI
- Bước 2: Ước lượng chế độ phát triển - độ phức tạp của sản phẩm: Organic, semidetached hoặc embedded

Một số kỹ thuật ước lượng

Ước lượng bằng COCOMO

- Bước 3: Tính nominal effort = $a \times (KDSI)^b$ person-months
 - Hằng số a, b phụ thuộc vào loại phần mềm: Organic (3.2 – 1.05); Semidetached (3.0 – 1.12); Embedded: (2.8 – 1.2)
- VD phần mềm organic có KDSI là 12:

$$\text{Nominal effort} = 3.2 \times (12)^{1.05} = 43 \text{ person-months}$$

Một số kỹ thuật ước lượng

Ước lượng bằng COCOMO

- Bước 4: Nhân nominal effort
vừa tính với 15 hệ số nhân
nỗ lực (effort multipliers)

Cost Drivers	Rating					
	Very Low	Low	Nominal	High	Very High	Extra High
Product Attributes						
Required software reliability	0.75	0.88	1.00	1.15	1.40	
Database size		0.94	1.00	1.08	1.16	
Product complexity	0.70	0.85	1.00	1.15	1.30	1.65
Computer Attributes						
Execution time constraint			1.00	1.11	1.30	1.66
Main storage constraint			1.00	1.06	1.21	1.56
Virtual machine volatility*		0.87	1.00	1.15	1.30	
Computer turnaround time		0.87	1.00	1.07	1.15	
Personnel Attributes						
Analyst capabilities	1.46	1.19	1.00	0.86	0.71	
Applications experience	1.29	1.13	1.00	0.91	0.82	
Programmer capability	1.42	1.17	1.00	0.86	0.70	
Virtual machine experience*	1.21	1.10	1.00	0.90		
Programming language experience	1.14	1.07	1.00	0.95		
Project Attributes						
Use of modern programming practices	1.24	1.10	1.00	0.91	0.82	
Use of software tools	1.24	1.10	1.00	0.91	0.83	
Required development schedule	1.23	1.08	1.00	1.04	1.10	

*For a given software product, the underlying virtual machine is the complex of hardware and software (operating system, database management system) it calls on to accomplish its task.

Một số kỹ thuật ước lượng

Ước lượng bằng COCOMO

- VD: phần mềm embedded với KDSI = 10
- Phiên bản nâng cấp: COCOMO II phức tạp hơn

Cost Drivers	Situation	Rating	Effort Multiplier
Required software reliability	Serious financial consequences of software fault	High	1.15
Database size	20,000 bytes	Low	0.94
Product complexity	Communications processing	Very high	1.30
Execution time constraint	Will use 70% of available time	High	1.11
Main storage constraint	45K of 64K store (70%)	High	1.06
Virtual machine volatility	Based on commercial microprocessor hardware	Nominal	1.00
Computer turnaround time	2 hour average turnaround time	Nominal	1.00
Analyst capabilities	Good senior analysts	High	0.86
Applications experience	3 years	Nominal	1.00
Programmer capability	Good senior programmers	High	0.86
Virtual machine experience	6 months	Low	1.10
Programming language experience	12 months	Nominal	1.00
Use of modern programming practices	Most techniques in use over 1 year	High	0.91
Use of software tools	At basic minicomputer tool level	Low	1.10
Required development schedule	9 months	Nominal	1.00

Kiểm thử phần mềm

- Phần mềm “xuất sắc”?
 - Chất lượng phần mềm thể hiện ở khả năng đáp ứng yêu cầu
 - Người làm phần mềm có trách nhiệm đảm bảo phần mềm chạy đúng
- > Chất lượng phần mềm phải được đảm bảo ngay từ đầu

Kiểm thử phần mềm

Thành viên nhóm SQA (đảm bảo chất lượng phần mềm):

- Đảm bảo các thành viên phát triển phần mềm đều làm việc với chất lượng cao:
 - Tại cuối mỗi workflow
 - Tới khi sản phẩm hoàn thiện
- Đảm bảo quy trình làm việc (VD: đáp ứng các chuẩn)

Kiểm thử phần mềm

- Quản lý nhóm SQA:
 - Độc lập với nhóm phát triển phần mềm
 - Không nhóm nào có quyền cấp trên
- Quản lý cấp cao của cả 2 nhóm mới có quyền quyết định:
 - Bàn giao sản phẩm đúng thời hạn nhưng có lỗi
 - Kiểm thử thêm và bàn giao trễ
- Quyết định phải dựa trên lợi ích của khách hàng và công ty



- Verification (xác minh): Cuối workflow
 - Kiểm tra tính đúng đắn và tương thích
- Validation (xác nhận): Cuối dự án
 - Kiểm tra đúng yêu cầu



- Kiểm thử thực thi (Execution-Based Testing)
- Kiểm thử phi thực thi (Non-Execution-Based Testing)



- 25

Kiểm thử thực thi (2)

Các trường hợp kiểm thử (test case):

- Viết trước khi có code và lưu lại sau khi kiểm thử
- Cấu trúc:
 - Tập dữ liệu đầu vào
 - Kịch bản các thao tác thực hiện
 - Kết quả mong đợi



- Correctness – Tính đúng đắn
- Utility – Tính hữu dụng
- Reliability – Tính tin cậy
- Robustness – Tính ổn định
- Performance – Tính hiệu quả



- Độc lập với sử dụng tài nguyên máy tính
- Đầu vào và đầu ra giống như trong tài liệu đặc tả

-> Kỹ thuật: Black-box testing

-> Kỹ thuật: Black-box testing



- Dễ sử dụng
- Chức năng cần thiết
- Giá chấp nhận được

- Dễ sử dụng
- Chức năng cần thiết
- Giá chấp nhận được



- Thời gian trung bình giữa 2 lần lỗi liên tiếp
- Thời gian trung bình để sửa 1 lỗi
- Chi phí và thời gian để khắc phục hậu quả của lỗi



- Phạm vi môi trường để sản phẩm có thể hoạt động ổn định là rộng
- Không được có lỗi khi đầu vào hợp lệ
- Không gặp sự cố khi hoạt động ngoài phạm vi môi trường

- Phạm vi môi trường để sản phẩm có thể hoạt động ổn định là rộng
- Không được có lỗi khi đầu vào hợp lệ
- Không gặp sự cố khi hoạt động ngoài phạm vi môi trường



- Đáp ứng các ràng buộc về bộ nhớ và thời gian
- Vấn đề tối ưu phần mềm

- Đáp ứng các ràng buộc về bộ nhớ và thời gian
- Vấn đề tối ưu phần mềm

Kiểm thử thực thi (9)

Ai là người thực hiện kiểm thử thực thi?

When a Developer
Finds a Bug

vs

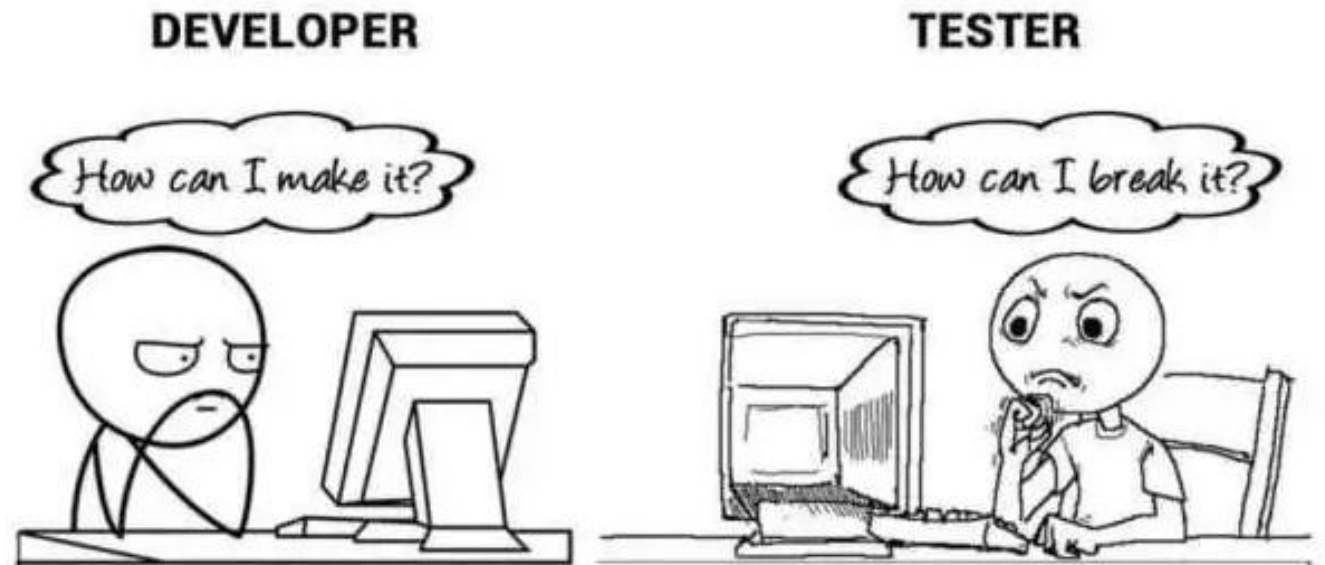
When a Tester
Finds a Bug



Codoid Innovations

Kiểm thử thực thi (9)

- "Programming is constructive, testing is destructive"
- > Lập trình viên không nên kiểm thử code của chính mình
- Tester kiểm thử code, lập trình viên sửa lỗi và kiểm tra lại
- Lập trình viên kiểm thử tài liệu





- 35



- Đại diện của workflow hiện tại và tiếp theo
- Đại diện nhóm SQA <- phụ trách (chair)
- Các thành viên đọc trước tài liệu và chuẩn bị danh sách:
 - Các vấn đề chưa hiểu
 - Các vấn đề có vẻ không đúng



- Kéo dài không quá 2 giờ
- Sửa lỗi ngay lúc đó không đảm bảo hiệu quả
- Không phải mọi vấn đề nêu ra đều là lỗi

-> Tập trung vào tài liệu, không phải người tham gia

-> Không sử dụng để đánh giá năng suất làm việc



- Người điều hành (moderator)
- Một đại diện workflow hiện tại
- Một đại diện workflow tiếp theo
- Một đại diện nhóm SQA



- Overview: Giới thiệu và phát tài liệu thẩm định tới các thành viên
- Preparation: Đọc hiểu tài liệu và xem xét danh sách lỗi từ các dự án trước
- Walkthrough: Rà soát lại tài liệu
- Rework: Sửa lỗi
- Follow-up: Moderator kiểm duyệt lại lần cuối



- “Checklist of potential faults”
- Thống kê lỗi:
 - Theo mức độ nghiêm trọng: “Major” hay “minor”
 - Theo kiểu lỗi

Kiểm thử phi thực thi (7)

Inspection: Thống kê lỗi

- So sánh tỉ lệ lỗi với các dự án trước
- Nếu số lỗi của một artifact nhiều bất thường -> cân nhắc làm lại từ đầu
- Bảo lưu số liệu cho workflow kế tiếp
- Không sử dụng để đánh giá năng suất làm việc: "~~Kill the goose that lays the golden eggs~~"



- Tác dụng: Phát hiện lỗi sớm
- Tìm được nhiều lỗi hơn chứng tỏ:
 - Đội rà soát/thẩm định làm việc hiệu quả hơn?
 - Đội phát triển làm việc kém đi?



- 44